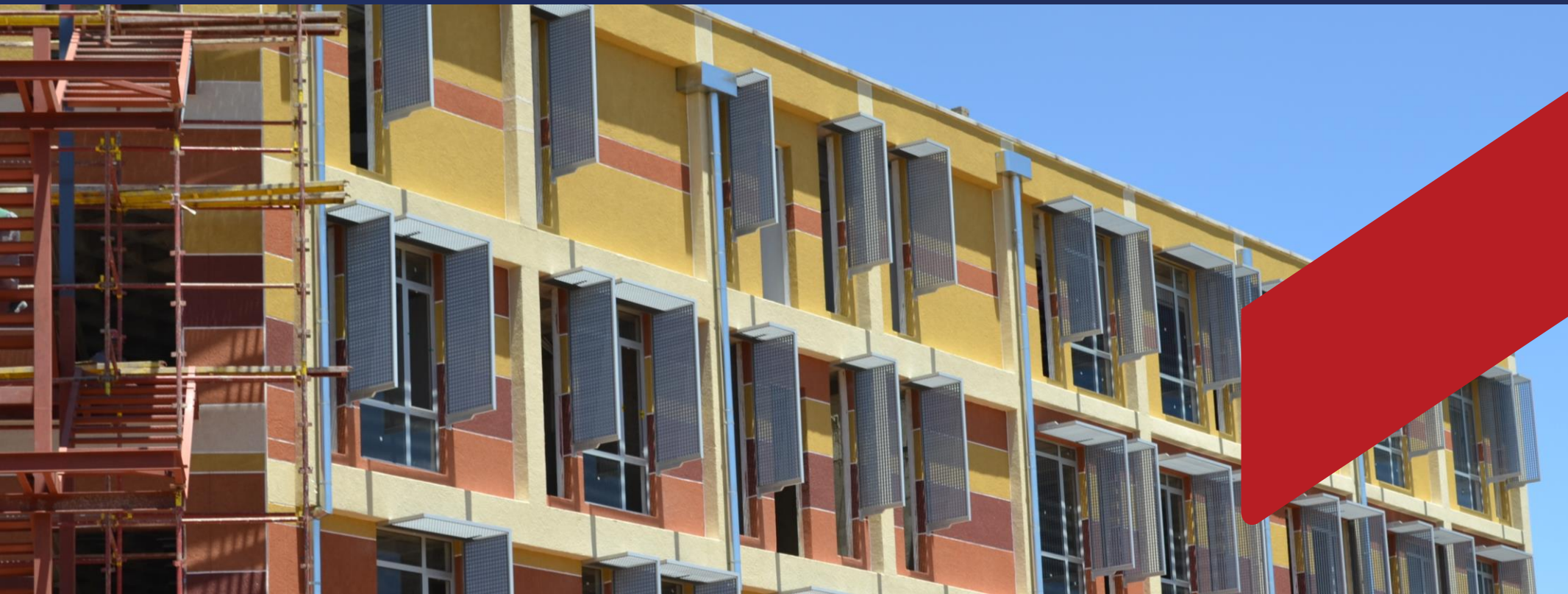
NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Lesson 2

Plotting and Presenting Your Course: The Test Plan



Overview

- Black-box functional testing: Developing test cases (Using use-cases), Operational profile-based testing
- Testing across quality attributes, Usability, Security, Compatibility, Accessibility & Deployment process
- Test Administration (Test planning, customisation of the test process, budgeting, and scheduling)

Why I Write Test Plans

- Writing a test plan gives you a chance to collect your thoughts, your ideas, and your memories.
- Writing a thorough test plan gives you a chance to crystallize that knowledge into a concrete way of tackling the tasks ahead.
- A test plan is an opportunity to communicate with your test team, your development colleagues, and your managers.
- The most intense discussion of what testing is all about often occurs when you hold a test plan review.
- Create two distinct types of documents: first a test plan, and then an accompanying document, set of documents, or entries in a test management tool that details the test cases.
- The test plan describes how we intend to implement the test strategy on a particular project. The test cases in the test suite provide the specific steps the testers will carry out during test execution, which are the tactics necessary to implement the plan.

How Many Test Plans?

- As the test manager, you have responsibility for the component, integration, and system test phases, with an aggressive beta-testing program during the system test phase

It is best to use separate plans for test subprojects that are distinct in one or more of the following ways:

- **Different time periods.** If the test planning, test development, test environment configuration, and test execution tasks for the subprojects start and end on different dates.
- **Different methodologies.** Detailed discussions of code coverage instrumentation and platform-independent automated GUI test tools don't really go together.
- **Different objectives.** Writing one plan each for the component test, integration test, and system test allows me to focus my thinking on each goal in turn.
- **Different audiences.** My test plan is not only my chance to inform my colleagues, my testers, and my managers of my vision; it is also a chance to discover their perspectives.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Using a Test Plan Template

- Overview
- Bounds
  - Scope
  - Definitions
  - Setting
- Quality Risks
- Proposed Schedule of Milestones
- Transitions
  - Entry Criteria
  - Continuation Criteria
  - Exit Criteria
- Test Development
- Test Configurations and Environments
- Test Execution
  - Resources
  - Test Case and Bug Tracking
  - Bug Isolation and Classification
  - Test Release Management
  - Test Cycles
  - Test Hours
- Risks and Contingencies
- Change History
- Referenced Documents
- Frequently Asked Questions

Overview

- The Overview section of a test plan allows you to introduce readers of the plan to your test project, including what your plan to test and the general test approach.
- You present a concise explanation of your goals, methodologies, and objectives.
- You keep this section brief.
- It is useful to include simple pictures or charts.
- You might want to illustrate concepts such as the architecture of the system under test, the decomposition or segmentation of the system for component or integration testing, or how this test effort fits into other test efforts that might precede, run concurrently, or follow.

Bounds

- In this section, you set boundaries for the test plan by discussing what you will and will not test, by defining important terms and acronyms related to the testing you plan to perform, and by determining where and in what context the test efforts associated with this test subproject will take place.

Scope

- When you describe the scope of your project, you tell the reader what you will and will not pay attention to during the course of the project.
- This compact form allows you to present a precise statement of scope.
- You don’t define each item at this point, as the details about each aspect of testing belong in the test cases themselves.

Definitions

- Testing, like other disciplines in the computer world, has its own terms and phrases.
- Therefore, you include a table of definitions in your test plans.
- Such a table can help to clarify terminology for those who are not experienced in the field of testing and can also help to ensure that everyone on the test team is operating from the same set of definitions.
- Don’t worry about using the ‘‘right’’ definition, as plenty of projects make use of their own unique definitions without any harm.

Setting

- This section of the test plan describes where you intend to perform the testing and the way those organizations doing the testing relate to the rest of the organization.
- The description might be as simple as ‘‘our test lab.’’
- In some cases, though, you might have testing spread hither and yon.

Quality Risks

- Summarize the quality risks here, given that your purpose is to communicate as well as to plan.
- However, if you know that people will read your quality risks analysis documents— or better yet, if all of the key audience members for the test effort participated in the quality risk analysis process— you can save yourself some work by referencing them.
- Also cross-reference the test strategy and the test environments against the various risk categories.

Proposed Schedule of Milestones

- Most of my test plans contain a schedule for the test project's major milestones.

Transitions

- For each test phase, the system under test must satisfy a minimal set of qualifications before the test organization can run tests effectively and efficiently.
- This section of the test plan should specify the criteria essential for beginning and completing various test phases (and for continuing an effective and efficient test process).
- Its usually refered to these as entry, continuation, and exit criteria, respectively, but some test professionals use the terms entry, suspension/resumption, and exit criteria or entry, stopping, and exit criteria.

Table 2-3 An Example Schedule from a Test Plan

MILESTONE	DATE
Test Development and Configuration	
Test plan complete	8/9/2010
Test lab defined	8/12/2010
FMEA complete	8/16/2010
Test lab configured	8/26/2010
Test suite complete	9/5/2010
Test Execution	
System test entry	9/2/2010
Cycle 1 Complete	9/16/2010
Cycle 2 Complete	10/3/2010
Cycle 3 Complete	10/13/2010
System test exit	10/13/2010

Entry Criteria

These criteria should address questions such as the following:

- Are the necessary documentation, design, and requirements information available that will allow testers to operate the system and judge correct behavior?
- Is the system ready for delivery, in whatever form is appropriate for the test phase in question?
- Are the supporting utilities, accessories, and prerequisites available in forms that testers can use?

System Test can begin when:

1. Bug tracking and test tracking systems are in place.
2. All components are under formal, automated configuration and release management control.
3. The Operations team has configured the System Test server environment, including all target hardware components and subsystems. The Test Team has been provided with appropriate access to these systems.
4. The Development Teams have completed all features and bug fixes scheduled for release.
5. The Development Teams have unit tested all features and bug fixes scheduled for release.
6. Less than fifty must-fix bugs (per Sales, Marketing, and Customer Service) are open against the first test release slated. (Fifty being the number of bug reports that can be effectively reviewed in a one-hour bug triage meeting.)
7. The Development Teams provide software to the Test Team three business days prior to starting System Test.
8. The Test Team completes a three-day "smoke test" and reports on the results to the System Test Phase Entry meeting.
9. The Project Management Team agrees in a System Test Phase Entry Meeting to proceed. The following topics will be resolved in the meeting:
    - . Whether code is complete.
    - . Whether unit testing is complete.
    - Assign a target fix date for any known "must-fix" bugs (no later than one week after System Test Phase Entry).

Entry criteria for SpeedyWriter

Continuation Criteria

- Continuation criteria define those conditions and situations that must prevail in the testing process to allow testing to continue effectively and efficiently.

System Test can continue if:

1. All software released to the Test Team is accompanied by Release Notes.
2. No change is made to the system, whether in source code, configuration files, or other setup instructions or processes, without an accompanying bug report. Should a change be made without a bug report, the Test Manager will open an urgent bug report requesting information and escalate to his manager.
3. The open bug backlog ("quality gap") remains less than fifty. The average time to close a bug remains less than fourteen days.
4. Twice-weekly bug review meetings (under the Change Control Board) occur until System Test Phase Exit to manage the open bug backlog and bug closure times.

Continuation criteria for SpeedyWriter

Exit Criteria

- Exit criteria address the issue of how to determine when the project has completed testing.

For example, one exit criterion might be that all the planned test cases and the regression tests have been run. Another might be that project management deems your results"OK," by whatever definition they use to decide such questions.

System Test will end when:

1. No changes (design/code/features), except to address System Test defects, occurred in the prior three weeks.
2. No panic, crash, halt, wedge, unexpected process termination, or other stoppage of processing has occurred on any server software or hardware for the previous three weeks.
3. No client systems have become inoperable due to a failed update during System Test.
4. The Test Team has executed all the planned tests against the GA-candidate software.
5. The Development Teams have resolved all "must-fix" bugs per Sales, Marketing, and Customer Service.
6. The Test Team has checked that all issues in the bug tracking system are either closed or deferred, and, where appropriate, verified by regression and confirmation testing.
7. The test metrics indicate that we have achieved product stability and reliability, that we have completed all planned tests, and the planned tests adequately cover the critical quality risks.
8. The Project Management Team agrees that the product, as defined during the final cycle of System Test, will satisfy the customer's reasonable expectations of quality.
9. The Project Management Team holds a System Test Phase Exit Meeting and agrees that we have completed System Test.

SpeedyWriter System Test exit criteria

Test Development

- In some cases, you find test teams rerunning tests that you created in previous test efforts.
- In other cases, you might have used a purely exploratory approach where you created the test data during testing.
- Typically, though, test projects may include some amount of work to design and develop various test objects, such as test cases, test tools, test procedures, test suites, automated test scripts, and so forth.
- Collectively, you may refer to these objects as test systems.

Test Configurations and Environments

- This section of the test plan is where you document which hardware, software, networks, and lab space you will use to perform the testing.
- For these various test systems, you will describe whatever important configuration details bear mentioning,

Test Execution

- This portion of the test plan addresses important factors affecting test execution.

For example, in order to run tests, you often need to receive items from the outside world, primarily resources (or funding for those resources) and systems to test.

- In the course of running tests, you will gather data that you must track, analyze, and report to your team, your peers, and your managers.
- In addition, you will run through distinct test cycles in each test phase.
- The level of detail required here varies from team to team and project to project.
- With your more senior test teams on well-run projects, you can leave some matters to the discretion of my crack team of testers.
- With junior testers, especially on chaotic projects, the more you can nail down the test execution realities during the planning phase, the more confusion you can anticipate and resolve ahead of time.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Resources

- In this section you identify the key participants in the test effort and the role they'll play in testing, along with any other resources not identified elsewhere in the plan.
- It is important to identify the external participants, the hand-off points, and each participant's contact information.

Table 2-4 A DataRocket Hardware Allocation Plan

TEST USAGE	SYSTEM [QTY]	NETWORK	WHEN	WHERE	OTHER[QTY]
Integration Test Phase					
Component interfaces	Engineering prototype [2]	Linux Samba, Sun Network File System, Windows 2003 Server	9/15–10/15	Engr lab	MS mouse, MS kbd, dual mon, USB mouse, USB mon, USB kbd, 3COM switch, Epson prn, Quantum HD, oscilloscope
Mechanical life	Engineering prototype [2]	None	8/1–10/1	Engr lab	None
Stress, capacity, volume	Engineering prototype [1]	Linux, NFS, Windows	9/15–10/15	Test lab	MS mouse, VGA mon
Performance	Engineering prototype [1]	Linux, NFS, Windows	9/15–10/15	Test lab	MS mouse, MS kbd, Dual mon, Quantum HD, solid state HD
System Test Phase					
MTBF demonstration	Validation prototype [4]	Linux	10/17–1/17	Engr lab	MS mouse, MS kbd, dual mon, MUX
Functionality	Validation prototype [2]	Linux, NFS, Windows	10/17–12/1	Test lab	MS mouse, MS kbd, dual mon
Stress, capacity, volume	Validation prototype [1]	Linux, NFS, Windows	10/17–12/1	Test lab	MS mouse, dual mon
Performance	Validation prototype [1]	Linux, NFS, Windows	10/17–12/1	Test lab	MS mouse, MS kbd, dual mon, Quantum HD, solid-state HD
Compatibility	Validation prototype [3]	N/A	10/24–12/1	System Cookers, Inc.	MS mouse [3], MS kbd [3], dual mon [3]
Environmental	Validation prototype [2]	N/A	10/24–12/1	System Cookers, Inc.	MS mouse [2], MS kbd [2], dual mon [2]

Test Case and Bug Tracking

- This section deals with the systems used to manage and track test cases and bugs.
- Test case tracking refers to the spreadsheet, database, or tool you use to manage all the test cases in the test suites and how you track progress through those tests.

(If you don't track the tests you plan to run, how can you gauge your test progress and coverage later on?)

- Bug tracking has to do with the process your team uses to manage the bugs you find in the course of testing.
- Since these systems form your principal communication channels inward to your own team, outward to other teams such as development, and upward to your management, you should define them well here.

Bug Isolation and Classification

- This section of the test plan is where you explain the degree to which you intend to isolate bugs and to classify bug reports.
- Isolating a bug means to experiment with the system under test in an effort to find connected variables, causal or otherwise.

For example, below are classifications such as the following:

- **Requirements failure.** The bug report concerns a failure of the system to meet its requirements. The appropriate party will resolve the problem. **Nonrequirements failure.** The reported bug is not covered by the system requirements, but it significantly affects the quality of the system in unacceptable ways. The appropriate party will resolve the problem. **Waiver requested.** The bug report does indeed describe a failure, but the programmers have requested a waiver because they believe that it will not significantly affect the customers' and users' experiences of quality. **External failure.** The bug report addresses a failure that arises from a factor or factors external to or beyond the control of the system under test. **Test failure.** The programmers believe that the test has returned a spurious or invalid error.

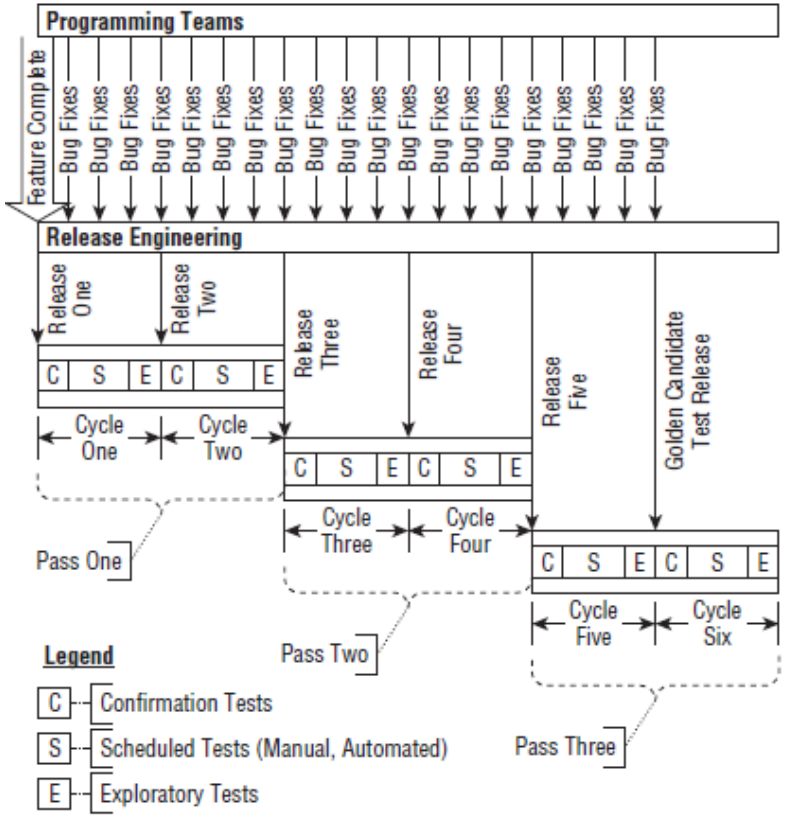
NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Test Release Management

- One of the major interfaces between the overall project and testing occurs when new revisions, builds, and components are submitted to the test team for testing.

On one project, a software development manager was emailing a new version of the software out to the entire project team, including the test team, every time any member of his team of programmers fixed a bug. We received a dozen test releases a day! I find that kind of anarchy unacceptable on my test projects, so in this section of the test plan I define the key elements of the test release process that can affect my effort.



Test passes, test releases, and test cycles

Test Cycles

- By a test cycle, it means running one, some, or all of the test suites planned for a given test phase.
- You can associate a test cycle with a single test release of the system under test, such as a build of software or a motherboard.
- Generally, new test releases occur during a test phase, triggering another test cycle.

For example, if test suites 3.1 through 3.5 are planned for a three-cycle system test phase, the first cycle could entail executing 3.1 and 3.2; the second cycle, 3.3, 3.4, and 3.5; and the third cycle, 3.1, 3.2, 3.3, 3.4, and 3.5.

- Any given test phase involves at least one cycle through the test suites planned for that phase.

Test Hours

- In some cases, you need to define the specific hours of testing.
- In addition, on some projects you can use multiple shifts to keep scarce resources humming 16 or 24 hours a day to accelerate the testing.
- In such cases, you include this section to define the specific hours and shifts that the team will use.
- You don't include this section unless you need it, though, and in many organizations it's unnecessary and maybe even counter-productive to have it; e.g., if your company has a flexible-hours policy you'll run afoul of that with your team by requiring work during certain hours.

Risks and Contingencies

These risks are possible outcomes or events that could make the test plan difficult or impossible to carry out.

- For any risk, you have four options:

Mitigation. Taking steps in advance that reduce the likelihood or impact of the event or outcome. **Contingency**. Being ready to act, should the risk become an actual event or outcome, to reduce its impact. **Transfer**. Getting another member of the project team or some other stakeholder to accept the impact of the risk should it become an actual event or outcome. **Accept or ignore**: Doing nothing.

- Let's look at some examples of project risks that can affect testing, along with various options for handling them.

Logistics or product quality problems. Uninstallable test deliverables. Excessive change in the requirements, design, or implementation of the system. Insufficient test environments, resulting in test delays. Unrealistic test environments, resulting in questionable test results. Unreliable test environment support. Gaps in test coverage revealed during test execution. Budget or staffing cuts reduce the total test effort. Debugging in the test environment reduces the total available test execution time.

Change History

- This part of the document records the changes and revisions that have been made to the test plan itself to this point.
- Specifically, you can assign a revision number and record who made the changes, what those changes were, and when the revision was released.

Referenced Documents

- As a rule, a test plan refers to other documents such as design specifications, requirements, the test suites, any quality risk analysis documents, and other pertinent information.
- Listing these documents in this section lets you avoid extensive repetition of their contents (which can create complications when these documents change).

Frequently Asked Questions

- On projects where you use neophyte test engineers and test technicians, you find that a frequently asked questions section is useful.
- Many of these questions entail describing the importance of the escalation process or some other delicate matter.
- That said, if you don't need this section, don't use it.
- As with the Test Hours section, using it inappropriately can waste your time and create problems.
- It can become a catch-all for any question anyone ever asked about testing, bloating your test plans into huge, unnavigable, and unmanageable documents.

Selling the Plan

Regardless of the template you use, after finishing a test plan, you want to obtain approval from project management and development peers. You can use the approach of attaching a formal sign-off sheet to the plan. The approach is a review meeting with all the involved managers. If your company tends to become mired in politics, you might need a sign-off to protect yourself. If such formalism is rare in your company, there are other ways to document approval, such as a circulation list that you keep. In any case, we recommend a tactful approach if you pursue a formal sign-off. Goose-stepping around the office with a document, requiring management signatures before you proceed with the test project, is not likely to endear you to your colleagues. You might be tempted to send the test plan via email, demanding that the recipients speak now or forever hold their peace. After failing to receive any criticisms or concerns, you might assume that everyone has read your plan and agrees with it. This assumption is typically false. Lack of response usually means that no one has read it. Review meetings are my favorite way to get people to read plans. One benefit of writing and circulating the plan is to provide a forum for discussing the test part of the project.

Clarity, Pertinence, and Action

- Some of the clients refer to long, dry test documents as shelfware.
- Many of us have seen too many test managers and senior test engineers spend too much of their time filling enormous binders that then sit on shelves, untouched during test execution.
- The style of writing matters, too.
- Avoid passive verbs. Don't say that something ''is to be done;'' instead, say exactly who is to do what and when.
- Defining roles and responsibilities keeps you out of the trap I saw one client's test manager fall into when she wrote a test plan that was just a collection of statements of principle and pieties (''Testing is good,'' ''Bugs are bad,'' etc.).
- Keep jargon, buzzwords, and so forth limited to those appropriate for the audience and use a Definitions section to promote further clarity.
- Keep your test documents practical, focused, and short.
- This approach to planning, customized to fit your particular needs, will help you write effective test plans that you can really use.

Case Study

On one project, my test team and I tested a browser-based program that provided home-equity-loan-processing capabilities to call-center banking agents.
The project was somewhat quirky, since we executed tests developed by another team and didn't have a chance to do a formal risk analysis.
However, this plan gives an example of how I applied the template shown earlier to a large in-house IT project. The document, ''Case Study Loan Processing Test Plan.doc,'' is available on the Basic Library page of www.rbcs-us.com. I thank my client on this project, a major midwestern financial institution, and the executive in charge of development for allowing me to share this example with readers of this book. I have purged all identifying and otherwise private information from this document. My client contact has asked that the organization remain anonymous.